Google Summer of Code

GSoC 2026 Proposal

Project: Create a new Concerto runtime for multi platform deployment

1. Organization: Accord Project

Project: Concerto runtime for multi platform deployment

Mentors: Ertugrul Karademir (@ekarademir), Jamie Shorten (@jamieshorten)

Project Size: Medium

Proposed Duration: 175hrs / 12 weeks

Category: Core Development

Difficulty: Hard

About Me

Name: Yash Kumar

Age: 19

Institution: IIIT Sonepat, B.Tech CSE (Sophomore)

Email: yashr17042006@gmail.com

GitHub: @yash-kumarx

Timezone: IST (UTC+5:30)

Available hours/week: 30-35 hours

2. About this Project

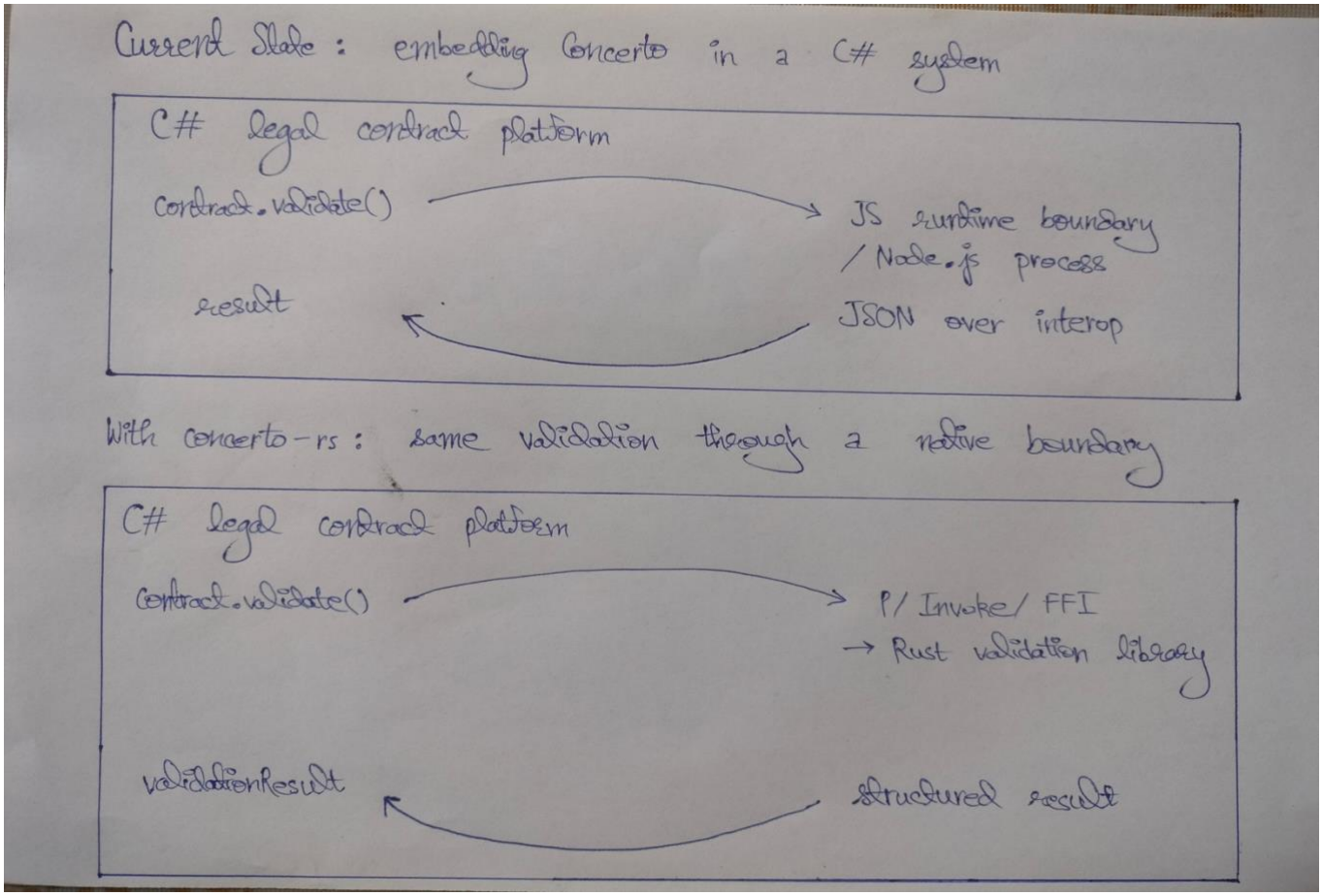
This project proposes a Rust-based runtime for Concerto, with entity validation as the first target and multi-platform deployment as the main goal.

Currently, Concerto's codebase has all its runtime logic coded in JavaScript. This is fine when the code will run strictly within the Node.js or browser execution context but it becomes awkward when the same validation logic needs to be reused from other environments such as Python, C#, or native systems. A Rust implementation is a good fit here as we could compile that to WebAssembly for use with JavaScript and provide access through the FFI for any non-JavaScript runtime (such as, e.g., Python). In other words, Concerto will only have to maintain one "implementation" of its core entity validation logic instead of having to expose it as two separate implementations.

Prior to applying for this project, I created a proof of concept for what I would like to accomplish. The POC already includes a Rust validation core, fixture-backed conformance tests, a WASM wrapper, a C FFI layer with a Python demo, and a command line interface for local testing. The suggested GSoC work would build on this by filling in the last semantic gaps in validation, increasing conformance coverage and making the WASM path a better way to integrate with the rest of the Concerto ecosystem.

3. The Problem

Concerto is a schema language for legal contracts. The runtime lives entirely in JavaScript today. Fine on Node and in the browser. The moment any other ecosystem tries to use it, there's a wall.



The proposed solution for this project consists of three main problems, as described below:

1) **Embedding cost**: Developers are required to cross a runtime boundary when they call the existing JavaScript validator from either C# or Python. This will typically be achieved through shelling out to Node or embedding a JS runtime engine. Data must be serialized back and forth. This makes it very cumbersome to integrate validation functionality as calling the JS validator incurs a lot of overhead and is not easily done in apps where JS cannot be embedded. The Rust library, once exposed via FFI, will offer a far less expensive integration solution and is far easier to embed into non-JavaScript-based applications.

2) **Performance at scale**: Legal platforms typically validate multiple times throughout the course of their operation. They validate documents during drafting, editing, template execution, and testing. Since most of the costs incurred in the existing JavaScript setup are due to excessive runtime overhead created by the existing JavaScript implementation rather than just the validation logic itself, moving from JavaScript to Rust will provide a more efficient way to perform repeated validation workloads.

3) **Browser and edge deployment**: A Rust core compiled to WASM will provide an easier way to deploy validator functionality into both browser and JS-facing environments without having to do a full reimplementation of the validator each time. This approach will also better meet the project's core requirements: one core implementation and multiple deployments.

That's why I believe this project is important. A Rust validation core wouldn't just be a rewrite for the sake of rewriting. It would make one version of the validation hotspot that can be used by many targets. I am focusing on entity validation first in this proposal because that is the part that most directly answers the real business question: whether a specific JSON instance is a valid instance of a given Concerto type. This priority was confirmed in the Accord Project TWG technical call: 'The hot path is validating whether a JSON object is a valid instance of a Concerto type. This priority was confirmed in the Accord Project TWG technical call: 'The hot path is validating whether a JSON object is a valid instance of a Concerto type

4. What is Concerto

Concerto is the data modeling language used in the Accord Project to describe the structure of legal contract data. A Concerto model specifies the types, properties, relationships based on inheritance, and any constraints on the values associated with the data relevant to an agreement before the data can be used with a template or processed downstream.

For this proposal, there are two different kinds of validation that must take place. The first is structural validation, which determines if a given JSON document is a valid Concerto model definition. The second is entity validation, which determines if a given JSON object is a valid instance of a specific Concerto type. The project will focus on entity validation because that will allow us to determine if the actual agreement data conforms to the schema as defined by the template in the most straightforward manner.

Thus, this will be the best place to start from an architectural standpoint. Once we have a solid foundation for entity validation, much of structural validation can be built on top of the same machinery by validating against the Concerto meta-model. Therefore, I consider entity validation to be the most significant first step in developing a Rust runtime: it solves the most pressing issues with the current implementation while also providing a foundation for other forms of validation to follow.

5. Current State of Existing Rust POCs

There are already two Rust repos in the accordproject GitHub organization related to Concerto: **concerto-validate-rs** and **concerto-rust**. I reviewed both before starting my own proof of concept.

- **concerto-validate-rs** focuses on validating Concerto models using the JSON AST / metamodel approach. While this model-based validation will ultimately help inform the creation of a set of entity-based validators, its primary purpose is to verify that the underlying structure being used to store or represent user data conforms with the structural model; therefore, the Concerto Validator does not directly solve the main use case for this initiative validating that user data is an instance of a user-defined Concerto type.
- **concerto-rust** is a broader prototype implementation of Concerto in Rust, including models, declarations, namespaces, imports, and validations associated with the metamodel. Consequently, it contains useful building blocks for creating a model management structure; however, it ultimately does not solve the entity validation problem.

I used both repositories for reference rather than templates; they allowed me to determine what has already been done and did not require me to follow either one. My POC takes a different approach: ModelManager owns the loaded namespaces and type resolution logic while validation is purely a function of the underlying state (i.e., the output of applying the method) keeping the core of this project easier to develop.

6. My Proof of Concept

6.1 What I Built

Before writing this proposal, I built a working proof of concept for the project. My goal was not to guess what the problems would be with the summer plan, I wanted to establish where we would run into difficulties when Concerto's validation logic was implemented in Rust.

Currently, my POC consists of a cargo workspace containing five distinct crates.

- concerto-core: the engine for validating data
- concerto-conformance: fixture-backed behavior checks
- concerto-wasm: a WASM wrapper for browser and JavaScript-facing use
- concerto-ffi: a wrapper for C ABI and a demonstration application written in Python using ctypes
- concerto-cli: a small command line tool for doing local development, viewing, and measuring performance

The primary architectural decision was to confine all of the validation logic to the concerto-core crate, while using the other crates as lightweight wrappers that act as thin wrappers around concerto-core. By following this approach, all of the validating functionality would be provided in one location with the different crates (CLI, WASM, and FFI) all exposing the same core functionality rather than moving away from a single shared implementation into separate implementations.

6.2 What the Core Already Handles

Currently, the POC covers the "standard" entity validation path, which includes the following:

- Required vs optional properties
- Unknown properties
- $class and requested type validation
- Primitive type validation (string, integer, long, double, boolean, and DateTime)
- String regex and length validators
- Lower and upper numeric bounds
- Enum validation
- Relationship validation
- Scalar validation
- Map validation
- Recursive object validation
- Inheritance-aware property collection across the entire inheritance hierarchy
- Checking for subtypes when the instance $class is more specific than the requested type.
- Collecting all validation errors prior to return.

One critical aspect of the design was differentiating between hard runtime/model errors and standard validation errors. For example, if a type that you have requested cannot be resolved because the namespace does not exist or inheritance resolution encounters a circularity then that

is clearly an error and thus a runtime/model error and therefore should return an error. Instance validation errors, such as missing/invalid fields, incorrect data types, or invalid enum values, would be collected into ValidationResult. That matches how callers are likely to use the runtime; therefore, runtime/setup validation errors should be returned at an earlier point in time than validation errors caused by invalid user input

6.3 The Hard Parts I Already Hit

The greatest challenge in the proof-of-concept was dealing with inheritance.

In order for a validator to properly validate a type, it must first gather properties from all superclasses in the entire supertype hierarchy. This becomes very apparent when looking at simple examples through to much more complex examples (for example, validating an Employee to Person to Entity). The proof-of-concept implementation resolved this in collect_properties_inner() by visiting each superclass recursively and keeping a HashSet of visited type names to detect circular references. The same recursive walk is used in is_assignable_to() to check whether an instance's $class is compatible with the requested type.

The second major difficulty was type resolution. Concerto has many different types of namespaces, imports, and both versioned and unversioned type names are found in practise. I kept this logic in the ModelManager class rather than scattering it throughout the validator class. ModelManager is responsible for managing the loaded namespaces, declaration lookups, import-aware resolution and fallback behaviour for versionless types. The validator is then able to validate based on already resolved model objects. This separation allows the core to be much simpler, and makes it easier to understand — and the wrapper layers are much simpler as well.

Another architectural choice that I made on purpose was to gather all of the validation errors before stopping at the first error found. In contract data work flows, fail-fast behaviour is less beneficial than providing all of the errors. For instance, if an instance of a type has three bad fields, users would need all three validation errors in one attempt.

6.4 Conformance, WASM, and FFI

I developed a conformance layer over and above the validator's existing unit tests. The current POC contains 12 related fixture-backed (passing) scenarios that demonstrate both valid and invalid behavior and functionality, including:

- missing required fields
- unknown properties
- child-as-parent validation
- unrelated requested types
- invalid datetime values
- validate regex constraints
- enum rejection
- relationship validation
- abstract type rejection
- validate scalar constraints
- validate map constraints

Because of this, we can measure how well the Rust validator functions. It is significantly easier to have discussions about parity and correctness of function when using fixture-based scenarios than it would be using manual reference examples only.

The POC already has both the WASM and FFI layers fully implemented and functioning. The WASM crate provides the same core validation to the browser and Node-style clients through both string-based and value-based APIs. The FFI crate supplies a C-compatible interface to the same model-manager/validator workflow, and I have implemented a demonstration of this with Python calling through ctypes. I believe these layers serve as proof that the core architecture can potentially be deployed across multiple platforms, not just as a Rust-only library.

6.5 What my POC Does Not Cover Yet

The main intentional shortcoming at this point in time is that the existing utilizes the JSON metamodel format for processing, not .cto source text. CTO parsing is still a stub.

There are also two things which the current POC has not attempted to address:

- There is no current connection to the upstream JavaScript runtime as a viable optional fast path.
- There is no current connection to the official upstream Cucumber-conformance path as well.

I consider these to be legitimate limits for this phase of development. The goal of the POC is simply to demonstrate that the Rust validation core, the type-resolution model, and the multi-platform-

wrapper approach are all functional (at this time). The POC was not intended to demonstrate "full runtime equivalence" prior to the start of the project.

6.6 Why This Matters for my Proposal

In my opinion, the POC does not demonstrate that the project's entirety has already reached completion; it instead demonstrates that the most difficult portion of the proposal has been practically explored to the level necessary to solidify the summer's program.

The POC has allowed me to do the following:

- verify that the validation hotspot has been developed and is capable of being implemented in Rust without any significant code duplication;
- re-use the core of the project across CLI, WASM, and FFI, free of duplicated logic;
- develop conformance-style tests based on the Rust implementation; and
- determine that the remaining work is not to determine if the solution is feasible, rather it is to provide additional coverage, tighten semantics and integrate the runtime more fully into the Concerto ecosystem.

As such, I chose to develop a POC prior to writing my proposal, so that I could rely upon actual implementation limitations instead of just making assumptions.

6.7 What I Have Already Verified

Currently, the proof of concept (POC) builds as a stand-alone Rust library. I have tried it out through multiple entry points:

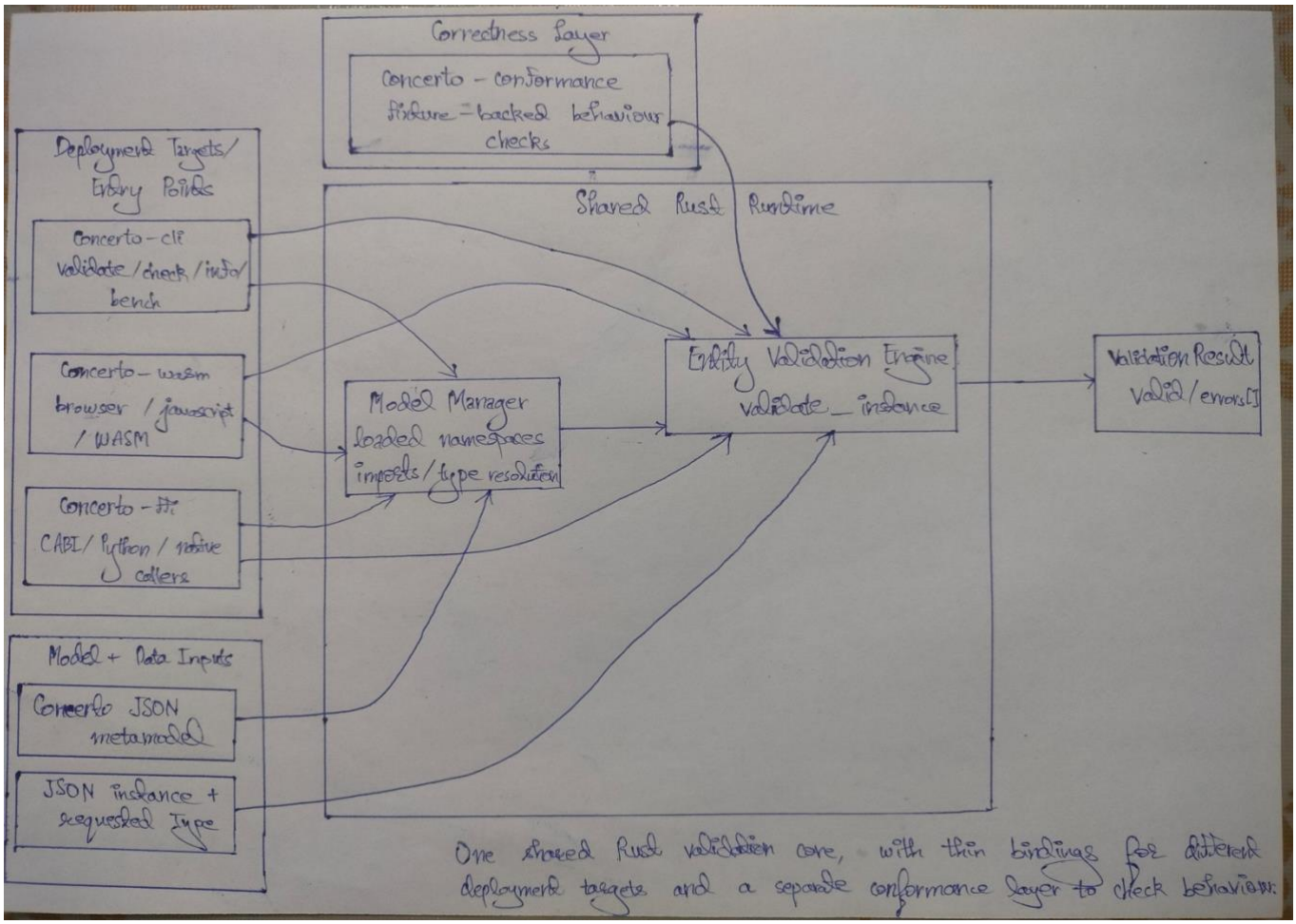
1. Rust API: Using only the validator, you can easily load a model, validate that you have loaded a valid instance of the model, and then provide a structured error if the loaded instance is not valid.
2. Command line interface (CLI): Using concerto-cli, you can validate any model/instance pair using files and provide the exact JSON path to where an invalid value is located.
3. Web assembly (WASM): Using a demo, the browser can load and validate any model/instance JSON in itself.
4. FFI: Using the C ABI, the project builds, and a demo written using Python ctypes can load and validate any model/instance JSON end to end.
5. Conformance testing: All fixture-backed scenarios for inheritance, enums, relationships, scalars, maps, dates, and edge cases pass.

I have confidence that the current architecture is both theoretically valid and functionally valid for each of the primary interfaces involved with this project.

7. Architecture and Design

7.1 Central Design Principle

The design I am proposing is centered around one reusable Rust validation core, with thin wrapper layers for different deployment targets.



The core idea is that validation logic should only exist in one place (concerto-core) and be called from all other target implementations. This keeps the project directed towards the true goal, which is to have a single runtime path that can be shared across many targets, rather than having separate validators for each platform.

7.2 Overall Architecture

The selection of the model-manager object as the only stateful object in the POC and in the GSoC project is a principal central design decision.

The ModelManager has the following responsibilities:

- loaded namespaces
- declaration lookup
- name resolution for type imports

- versioned and versionless name handling

The only way to validate the data in the model is to pass the loaded model manager, the JSON data and the type requested to the model manager to get either an error due to a hard runtime error/model error or a structured result.

This separation is significant for two reasons:

- It allows the validator to not be distracted by the book-keeping of namespaces/imports and concentrate on the validation logic
- it keeps the layers that wrap around the validators to a minimum, since the CLI, WASM, and FFI all only have a handle to a model manager and can all call the same validation entry point.

7.3 Model Loading and Type Resolution

The current Proof of Concept (POC) utilizes Concerto's JSON meta model form. Models are loaded into the ModelManager which stores models by namespace and resolves model declarations by either fully qualified name or contextually relative name.

Type resolution is not solely based on splitting the name of an object into string segments, it also has to account for the following:

1. Local Declarations

2. Explicitly Imported Names

3. Wildcard Imported Names

4. Namespaces Versioned

5. No Versioning Support for Lookups

I believe that this type of logic belongs in model management rather than being scattered throughout the validator. If the validator must create a separate mechanism to perform name resolution at every instance of a model, then the validator design will be difficult to manage and reason about, especially when nested object references and/or inheritance hierarchies are introduced.

7.4 Validation Flow

The overall process for validation in concerto-core is as follows, beginning with the entry point validate_instance() and collecting all failures into ValidationResult via the ErrorKind enum::

1. Determine the $class of the requested type and the instance to validate
2. Determine whether the type of the instance is assignable into the requested type
3. Create a complete set of properties for the instance using the inheritance hierarchy of the instance
4. Confirm whether required properties and unknown properties exist for the instance

5. Check each property value for validity according to its type and defined constraints
6. Return all of the validation errors that have been found by the validator back to the caller, which matches the explicit preference expressed in the Accord Project TWG technical call: We want all the errors reported at once so people can fix everything in one pass.

The order in which these steps are executed is critical; in particular, step 3 must be performed before the beginning of any actual field validation. This ensures that proper validation can be performed for fields (that are required) defined on any superclass of the instance. In addition, proper validation will also be able to be performed on any child instances; validation can properly validate that the type of a child instance is correct with respect to the requested parent type, but must also ensure that type mismatches (i.e., unrelated instances) are correctly reported.

Another design choice is that all errors for instance validation are collected into a single data structure before returning them to the caller. In practice, this will be very beneficial for contract data workflows as it allows callers to receive every validation error in a single pass rather than discovering failures one at a time.

7.5 Error Model

The error model is divided into two classes of error:

1. Hard runtime/model errors
2. Normal validation failures

Hard runtime/model errors include condition(s) like:

- The requested type could not be resolved
- A namespace is missing
- There is circular inheritance in the model

These types of errors should generate a real error because you cannot continue validation in any meaningful way.

Normal validation failure(s) include conditions like:

- Required fields are not provided
- Received JSON type is not compatible
- Provided enum value(s) do not match defined values
- Relationship between records do not match
- At least one constraint has been violated

These types of errors should be aggregated into a structured validation result because they show that invalid data was submitted through the user; however, the runtime environment did not fail as a result of the invalid data submitted.

I think this differentiation is crucial for true integration between systems/components. If the caller is not clear on if there was an issue with their configuration vs. if their submitted contract data is simply invalid, then they will be at a disadvantage.

7.6 Multi-Platform Wrapper Design

The wrappers are purposely designed to be very light in weight.

The CLI is intended for use with local testing, inspection, and benchmarks. WASM is the highest-priority integration target — as stated in the TWG technical call: 'The browser and Node.js target is the one that's going to get the most usage initially.' WASM provides access to the core Rust validator through WasmModelManager, which exposes validateInstanceValue(JsValue, &str) and addModelValue(JsValue) for JavaScript-native callers, enabling validation in both browser and Node.js environments without recompiling any validation logic. FFI provides access to the the same Rust core through a C-compatible boundary so it can be viewed by applications using Python, as well as other native environments.

These wrappers are intended to convert the input data into a form compatible with Concerto-core, call the same validation application programming interface, and return the outcome in a format that can be understood by the application.

The real multi-platform aspect of the overall design will come from the fact that the value of this project comes from the fact that one core runtime can be reused cleanly, across many types of borders, without having to reimplement the validator.

7.7 Why This Architecture Fits the Project

This architecture will be well suited for my GSOC project for three reasons:

1. This architecture promotes the clarity of the critical path. Since the validation core represents the most significant portion of time spent on this project, the design should maintain this as the focal point of the project.

2. The architecture allows for a natural scale of targets. Once the validation core has been created by the validation process, the other components will primarily require the addition of wrapper code to create new means of deploying the core as opposed to creating new validators.

3. The architecture allows for future runtime growth. The POC for this project is focused on entity validation, but the overall architecture will allow for future runtime expansion of the validation core without requiring the validation core to be discarded.

Thus, the primary architectural goal of this project is not to create the largest possible Rust code base, but rather to create an easily integrated (with the overall Concerto ecosystem) validation core that is both reusable and accurate.

8. Integration Plan - How my POC will become Real GSoC Work

My poc (proof of concept) offers several significant justifications that will help provide the foundation for the eventual integration of GSoC work into the Accord Project as a whole:

- The ModelManager: validate_instance architectural model is being validated from end to end in an actual "real world" scenario with actual "real world" data.
- WASM (WebAssembly modules) bindings (using wasm-bindgen) are achievable, as evidenced by a functional demo available in browsers now.
- FFI (Foreign Function Interface) mechanisms in a C ABI are achievable, as evidenced by a functional demo for Python.
- There are no fundamental architectural limits affecting the ability to add new conformance test fixtures without rewriting the test harness.
- The GSoC work should synergistically complement the foundation now established with the POC and lead closer toward production-ready code.

The GSoC development would extend where this work would go and allow me to develop from my GitHub repo into the Accord Project repo structure.

- Update the upstream with my work. Right now the GSoC work is housed in my repo. During GSoC, my work can move into the Accord Project repo structure so that the development is completed in the context of the Accord Project instead of as a prototype.
- Broaden conformance coverage. I have a small number of fixtures to illustrate the intended direction; however, I don't have a large enough number of fixtures to encompass all of the various conditions that the project is anticipated to encounter.
- Use the upstream conformance style. Each .feature file will describe a validation scenario in human-readable Gherkin — for example, 'Given a Person model, When I validate an instance missing the firstName field, Then the result should contain a MissingRequiredProperty error at path /firstName.' The Rust step definitions drive concerto-core directly, with no intermediate layer. This means any future runtime in another language can reuse the identical .feature files as a compliance target, which was identified in the TWG call as a core requirement: 'We need a conformance test suite that all runtimes have to pass.
- Build the WASM layer into an actual integration path. My goal for GSoC is to build the current WASM build into a functioning integration path for the current Concerto runtime that is using JavaScript. My objective is that future work will be easier to do with this than if it were not present.

Clean up the concerto-core crate for public consumption, including the APIs, packaging, documentation and repository integration. The concerto-core crate currently provides reusable validation functionality; therefore, the GSoC period will provide an opportunity to focus on accessibility of these packages.

9. Deliverables with Milestones

Major Deliverables

1. Rust validation core

Core components to include Rust validation component and improved concerto-core crate encompassing the principal Concerto Models entity-validation pathway in a JSON-based metamodel context.

Consisting of :

- Complete Inheritance Chain Validation
- Required and Optional Property Validation
- Primitive Types, Enums, Relationships, Scalars, and Maps Validation
- Structured Validation Results with Enhanced Error Reporting
- Regression Tests for Complex Validation Scenarios

Milestone: stable enough to utilize as a basis for integration versus still behaving like an experiment.

2. Cucumber-Based Conformance Coverage

Improvements to the current conformance layer around the Rust validator to provide a serious basis to build on that already contains fixture-based scenarios, in order to develop further and more realistic coverage during the project.

Consisting of:

- Expanded Valid and Invalid Scenarios
- Regression Scenarios for Instances of Bugs Found During Implementation
- Alignment with Expected Workflow for Project Involving Conformance Verification
- Gherkin .feature files covering all validation scenarios, reusable by any future Concerto runtime as a shared compliance target

Milestone: checking through realistic validation scenarios versus verifying through custom unit test cases.

3. Integrating the WASM

The objective is to have a WASM-backed path for using the Rust validator from JavaScript-based environments. This path will have:

- stable bindings to the Rust validation API via WASM
- loading of model and validating instances using JavaScript-side inputs
- provided end-to-end examples or smoke tests for integration validation

Milestone: the Rust validation core is usable from a JavaScript workflow and not limited to standalone Rust-only testing

4. Native interface for non-JS environments

The goal of this milestone will be to have a maintained native boundary around the same Rust validation core for callers outside of the JavaScript ecosystem. This path will provide:

- a small interface with the Rust validator that is C-compatible
- one demo path that will run as a non-JS language, such as Python using ctypes
- a thin layer which reduces the chance of this interface becoming a second implementation

The ability for other languages to use the same validation core without relying on a JS runtime will be demonstrated in this milestone.

5. Project Cleanup and Documentation

A final goal of this project is to have it in a clean and logically organized state that will make it easier for potential new developers that want to support this project by reviewing, running, and building off of it. This will occur via:

- providing clear build and usage instructions
- producing better examples of running the validator
- cleaning up rough prototype edges within the repository and API surface
- producing a clear summary of what has been completed versus yet to be completed

The end result of this milestone will be that developers can more easily navigate all aspects of the project.

Stretch Targets

If all the base milestones were completed ahead of schedule, and I had a stable path validated and integrated-then, instead of adding additional scope to your original milestone target(s), I would direct to use available time on additional work.

Some possible stretch target areas include:

1. Increased run-time coverage than what is currently available relative to the validation work you have completed.
2. Reduced inconvenience of entering data into your system via methods other than JSON, JSON metamodel-based input.
3. Begin CTO text parsing using a pest grammar, enabling round-trip from .cto source files to the JSON metamodel without depending on the TypeScript toolchain.
4. When you have a stable API surface area, assist in packaging and publishing your application(s).

10. Week-by-Week Timeline

Community Bonding Period

- Review the current POC with mentors and agree on the exact project scope for the summer.
- Confirm which parts of the current implementation are strong enough to keep, and which parts should be reshaped to fit upstream expectations.
- Study the existing upstream conformance setup and identify how the current Rust fixture-based approach should be aligned with it.
- Finalize the repository structure, review workflow, and the expected integration target for the WASM path.

Phase 1: Strengthen the Rust validation core

Week 1

- Clean up the current concerto-core API and align it with the final agreed project scope.
- Review the current validation path carefully and list the remaining semantic gaps relative to the expected Concerto behavior.
- Add regression tests for any already-known weak spots before making larger changes.

Week 2

- Integrate the Cucumber conformance harness. Convert the existing 12 fixture-backed scenarios into Gherkin .feature files, with Rust step definitions wiring directly into concerto-core. This establishes the official conformance structure the Accord Project expects all runtimes to share.
- Recheck required-property behavior, unknown-property handling, and recursive validation of nested object references.

Week 3

- Expand Cucumber scenario coverage to cross-namespace resolution edge cases, versioned vs. versionless namespace fallback, and abstract type rejection. Add regression .feature files for any edge cases discovered during Week 2 integration.
- Tighten error reporting so failures remain clear and structured across both Rust callers and wrapper layers.

Week 4

- Strengthen the conformance layer by expanding fixture-backed coverage for both valid and invalid cases.
- Use this week to harden the core and prepare for the midterm evaluation, rather than opening new architectural risk.

Expected midterm state:
A stable Rust validation core with stronger coverage for the main entity-validation path, backed by meaningful conformance-style scenarios.

Phase 2: Multi-platform integration

Week 5

- Refine the WASM-facing API so model loading and instance validation can be called cleanly from JavaScript-facing environments.
- Keep the wrapper thin and avoid duplicating validation logic outside concerto-core.

Week 6

- Test the WASM path end to end with realistic model and instance inputs.
- Fix integration issues found during that process, especially around input conversion, error propagation, and API shape.

Week 7

- Strengthen the native interface layer around the same Rust core.
- Keep the C-compatible boundary small and make sure at least one non-JS caller path remains working and easy to demonstrate.

Week 8

- Expand conformance and regression coverage again using what was learned during the integration phase.
- Use any remaining time here as buffer for overflow from the WASM or native-interface work.

Expected end of Phase 2:
The Rust validation core is no longer only a standalone library. It is working through WASM and through at least one native embedding path, while still sharing the same underlying validation logic.

Phase 3: Integration, polish, and upstream readiness

Week 9

- Wire the WASM build into the existing Concerto JavaScript runtime as an optional validation backend, replacing the equivalent JS call path in at least one integration test.

Verify that the WASM bundle size is acceptable for browser deployment and profile any hot spots using wasm-pack output metrics.

Week 10

- Close all open semantic mismatches between the Rust validator and the reference JavaScript runtime, verified by running both against the same Cucumber `.feature` files.

- If core work is stable, begin stubbing a `pest` grammar for `.cto` source text as the first step toward CTO round-trip parsing.

Week 11

- Write `rustdoc` documentation for every public item in `concerto-core` and publish the WASM package to npm under a scoped Accord Project namespace.
- File a pull request against the upstream Accord Project repository with the complete runtime, Cucumber suite, and integration path, ready for mentor review.

Week 12

- Reserve the final week for mentor feedback, bug fixes, final stabilization, and packaging of deliverables.
- Document completed work, known limitations, and the most useful next steps beyond the coding period.

Priority and Risk Management

Some of these tasks will naturally overlap in practice. If integration work takes longer than expected, I would prioritize the following in this order:

1. Validation correctness in concerto-core
2. Broader conformance coverage
3. A working WASM integration path
4. Native interface polish and stretch goals

11. About me

I am Yash, a second-year student. I study B.Tech in Computer Science Engineering at IIIT Sonepat. After learning Java & C++ (both languages) and have had experience (months) trying to understand Rust through both private projects and Open Source contributions, I will be honest with you — most of my experience has come from writing real-world projects in Rust, but the part where I learned the most was actually reading real-world projects first and then trying to understand how each API, error handling, and module structure was designed.

I have already contributed to several rust issues in The Rust Foundation, which has been useful for learning how to work with review feedback, unfamiliar code, and long iteration cycles.

- **rust-lang/cargo PR issue #15305 / PR #16653**
  In order to enhance Cargo's unstable-edition error message, the required rust-version will now be indicated rather than just instructing the user to upgrade to a newer version of

Cargo. I was able to gain experience on a user-facing bug in a significant production Rust codebase thanks to getting this merged.

- **crate-ci/cargo-cargofmt issue #56 / PR #61**
  This work is about grouping dotted TOML keys with the same root together, following TOML's guidance that out-of-order dotted keys are discouraged. My PR implements that formatting behavior and is still under review.
- **crate-ci/cargo-cargofmt issue #54 / PR #59**
  This task includes table hierarchy sorting so that tables like [package] and [package.metadata] appear in a hierarchical order rather than a first-come fashion. This PR is still ongoing and already has some formatter-specific review comments.
- **crate-ci/cargo-cargofmt issue #8 / PR #51**
  This PR implements support for comment_width and wrap_comments, which are part of the formatter's Rust-like configuration story. It is still open and is going through substantial review discussion.
- **rust-lang/rust-clippy issue #16520 / PR #16549**
  This PR adds a new Clippy lint around replacing collection reassignment with a more efficient .clear()-style pattern. It is still open and has gone through multiple rounds of detailed review, especially around edge cases and test coverage.

With respect to this particular project, I did not want an abstract application plan. I first spent some time creating a proof of concept in order to fully understand the real world engineering challenges prior to submitting a 12-week plan for this project. By actually doing the work instead of creating an application, I have learned more about inheritance management, type resolution, wrapper design and validation semantics than I would have learned by simply writing a proposal. I believe this is the strongest reason I'm a good fit for this project; I won't be starting from nothing and therefore I will have some level of familiarity with the difficult parts of development for this project.

12. Time Commitment and Availability

I will be available full-time during the GSoC coding period and expect to spend around 30 to 35 hours per week on the project.

I do not have any internship or major work commitment scheduled during that time. My semester exams will end before the main coding period (i.e. start of may), so they should not interfere with regular progress once the project begins.

I am based in IST (UTC+5:30), which gives overlap with both European and evening US time zones. That should make regular mentor communication practical.

Keeping it real means that some periods are going to have more focus on debugging, reviewing issues and clearing stuff up than actually building visible features (particularly true when integration begins). I'm ok with that.

My proof of concept created for this proposal can be found at the following link:
https://github.com/yash-kumarx/concerto-validate-rs